

MAE 3780: Mechatronics Robot Project Final Report

Term: Spring 2026

Team #60

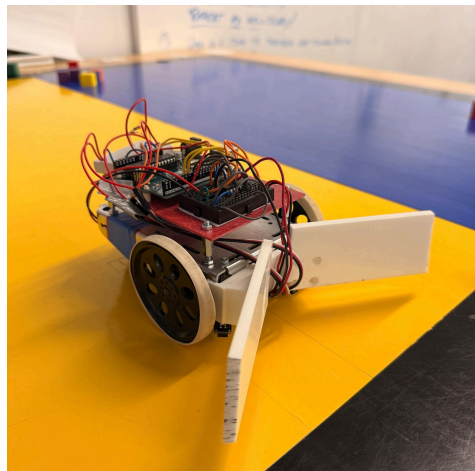
By:

Ethan Hernandez (emh236)

Huxley Holcombe (hmh96)

Yussef Oufkir (yro2)

Robot Design and Strategy Overview



In designing our robot, we placed strong emphasis on simplicity to minimize points of failure and keep assembly straightforward. Our cube-capturing mechanism consisted of a funnel mounted at the front of the chassis, with a wide outer opening to maximize the robot's collection area. The robot was built on a Boe-Bot chassis, with an Arduino board and two mini-breadboards secured on top using standoffs, with a 9V battery mounted directly underneath the Arduino. A AA battery pack was attached to the underside of the chassis with duct tape, alongside two continuous rotation servo motors that drove the wheels. Thick rubber bands were wrapped around the wheels to improve traction. The

Boe-Bot's rear tail wheel, a 1-inch polyethylene ball attached via a cotter pin axle, provided stability and allowed the robot to turn freely. Two QTI sensors were mounted at the front underside of the chassis, one on each side, positioned close to the ground via standoffs for border detection. A 3D-printed mounting bracket at the front of the chassis supported the laser-cut acrylic arms forming the funnel.

Regarding the electrical design, one mini-breadboard housed the motor driver circuitry while the other was dedicated to the QTI sensor circuit. The motor circuitry employed two L9110H H-bridge motor driver ICs, one for each wheel motor. Each H-bridge received two digital input signals from the Arduino (pins 8–11), which controlled the direction of current through the motor. The H-bridges were powered by the AA battery pack, with their output pins connected directly to the motor leads of each wheel. Each QTI sensor had a 10 k Ω pull-up resistor added across its W and R terminals. Each sensor's W pin was connected to 5V power from the Arduino and its B pin to ground, with the R (signal) pin connected to Arduino digital pins 6 and 7 for the left and right sensors respectively. The Arduino reads these pins directly as binary inputs: a HIGH reading indicating a black surface (the border) and a LOW reading indicating the yellow and blue playing field.

During operation, the robot continuously reads the two QTI sensors to check whether either is detecting the black border. If neither sensor detects the border, the robot simply drives forward, funneling any cubes it encounters toward itself. If a sensor detects the black border, the robot executes an escape routine depending on which sensor triggered it: if only the left sensor detects the border, the robot briefly reverses and turns right at about a 30 degree angle; if only the right sensor triggers, it reverses and turns left at the same angle; and if both sensors trigger simultaneously, indicating a head-on approach to the border, the robot reverses for longer and executes a sharper, almost

180 degree turn to reorient itself back toward the center of the field. This cycle repeats indefinitely for the duration of the match. Cube collection is essentially opportunistic and relies on the robot continuously bouncing around the board by avoiding the black border.

Design Process Reflection

Our final robot was largely similar to our initial sketches from the first milestone, with one notable exception: a spinning brush that was to be attached at the end of the funnel. This mechanism was initially included as a way to better guide cubes into the funnel and keep them inside once collected. The idea was ultimately scrapped during milestone testing due to budget constraints, difficulty of implementation, and doubt that the mechanism would be more effective than simpler, cheaper alternatives.

During milestone 2, the main chassis and motor circuitry were assembled, and the motion control code was organized into helper functions that could be easily built upon in future milestones. Milestone 3 proved to be the most challenging phase of the design process. At this stage, a color sensor was added to the underside of the chassis to detect whether the robot was on the blue field, yellow field, or black border, using period calculation to distinguish between surface colors. During testing, we encountered difficulty reliably differentiating between the blue playing field and the black border, as their measured periods were relatively close and sensitive to changes in ambient lighting. Since we were unable to meet the milestone objectives reliably with the color sensor alone, we opted to install two QTI sensors to handle border detection instead. With the QTI sensors in place, border detection became reliable and consistent. Milestone 3 ultimately laid the foundation for the border-avoidance logic used in competition.

Following milestone 3, we decided to remove the color sensor entirely, as distinguishing between the blue and yellow halves of the field was no longer relevant to our strategy. This simplified both the code and the circuitry considerably. Milestone 4 was the primary driver for refining the cube collection mechanism. To initially complete the milestone, we constructed a simple cardboard funnel to collect the required five cubes. We subsequently replicated the design using a 3D-printed mounting bracket and laser-cut acrylic arms to improve the funnel's stability and rigidity. However, we encountered a significant obstacle close to competition day: slight mismeasurements had caused the robot to exceed the allowed 8"x8" size limit, requiring us to cut the tip off the funnel. The original tip featured inward-curving edges designed to retain cubes during turning maneuvers. Unfortunately, due to time, budget, and dimensional constraints, we were unable to replicate this curved design, and proceeded with an open funnel on competition day.

Competition Analysis

Our robot ultimately performed moderately on competition day. We placed 5th out of eight in the round robin portion of the tournament and did not advance to the next

round. A clear strength of our robot was its reliability, owing to its simple design. While other robots exhibited inconsistent behavior or drove off the field entirely, ours consistently drove forward, collected cubes, and bounced off the black border as intended.

A significant weakness, however, was the robot's difficulty retaining collected cubes. Due to the open funnel design and lack of friction on its interior surfaces, the robot would frequently lose a large number of cubes when making turns. As a result, despite navigating the field reliably, the robot struggled to accumulate a competitive cube count.

We also encountered issues when the robot collided with a stationary opponent: since the robot has no means of detecting other robots or obstacles, it would become stuck against them with no way to escape, missing valuable time that could have been spent collecting cubes. A recurring mechanical issue arose in the corners of the field. When approaching a corner, the left QTI sensor would detect the border and trigger a rightward bounce, only for the right sensor to then detect the opposite border and trigger a leftward bounce, trapping the robot in a repetitive left-right oscillation until it narrowly escaped.

The most unexpected incident occurred in one round, when the robot became stuck rotating in place in the middle of the yellow portion of the field for no apparent reason, rendering it completely unable to collect cubes and conceding the round. We tested the QTI sensors immediately after and could not replicate the issue, and it did not recur for the remainder of the competition. Overall, the robot demonstrated a highly reliable and consistent strategy, but the open funnel design proved to be a critical limitation in retaining cubes during competition.

Conclusions

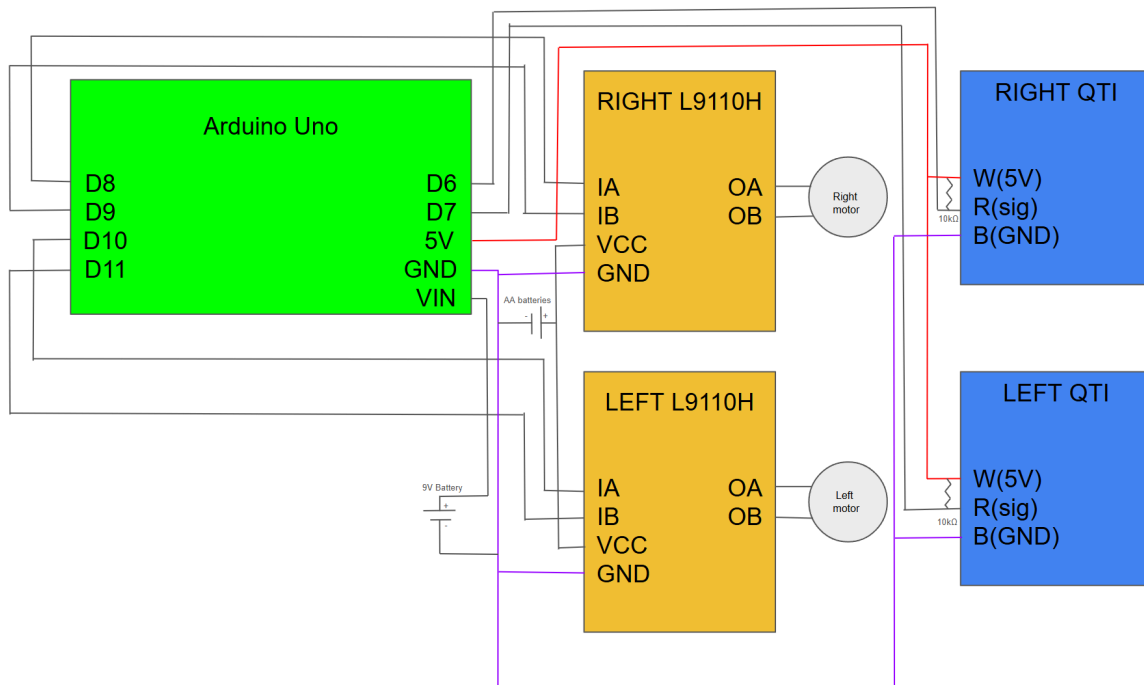
This project was a valuable exercise in iterative engineering design, and the robot's consistent performance on competition day validated our emphasis on simplicity and reliability. However, the competition exposed several clear areas for improvement.

The most impactful change would be to the cube containment mechanism. A closed collection area at the base of the funnel would prevent cubes from escaping during turns. Incorporating the ultrasonic sensor to detect opposing robots would also be a meaningful addition. As it stood, the robot had no way to recognize or avoid other robots, causing it to become stuck against stationary opponents and lose valuable collection time.

For future students, our advice is simple: design and test proactively. Decisions made early can have significant consequences later, as we experienced firsthand. Verify all dimensions against the competition rules before finalizing any parts, and invest time early in refining your competition strategy. The earlier you have a functional robot, the more time you have to make it competitive.

Appendix A: Bill of Materials

<i>Item #</i>	<i>Component</i>	<i>Source</i>	<i>Quantity</i>	<i>Unit Cost (\$)</i>	<i>Total Cost (\$)</i>
1	Mounting Bracket	3D Print - RPL	1	\$12.86	\$12.86
2	Funnel Arms	Laser Cut - RPL	2	\$3.29	\$6.58
3	Cardboard Sheet	Sourced from Home	1	\$2.00	\$2.00
				Total (\$)	\$21.44

Appendix B: Circuit Diagram

Appendix C: CAD Files and Drawings

Figure 1: View of Cube Capturing Mechanism

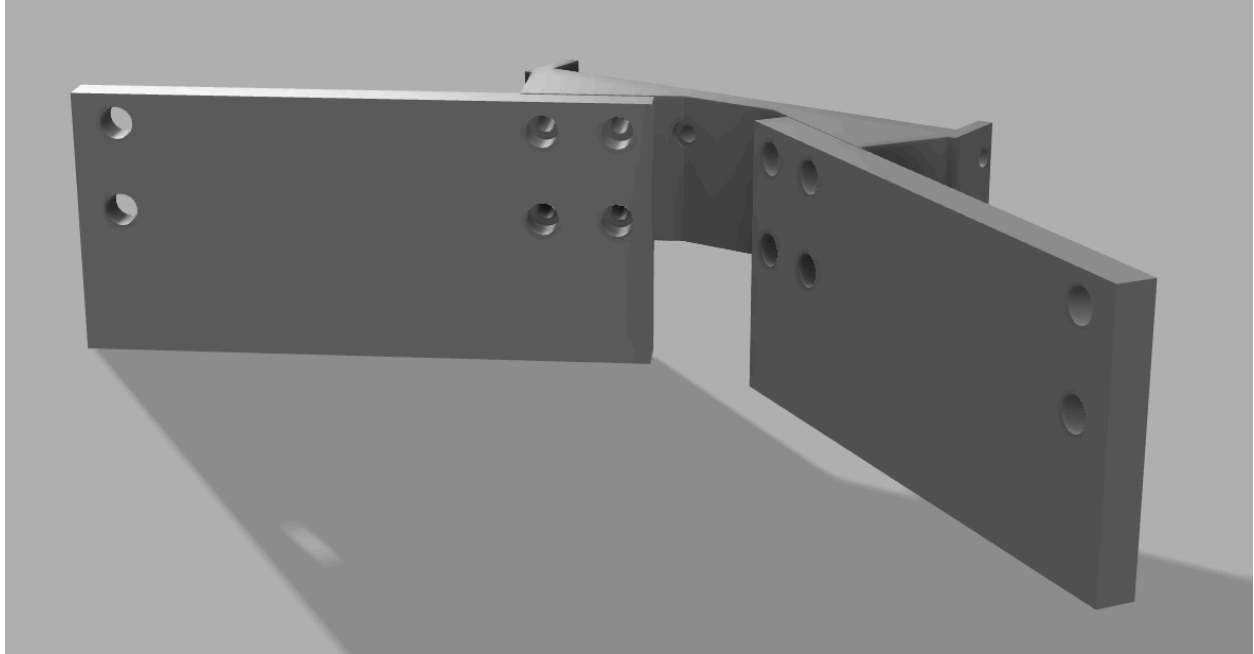


Figure 2: Overhead View of Cube Capturing Mechanism

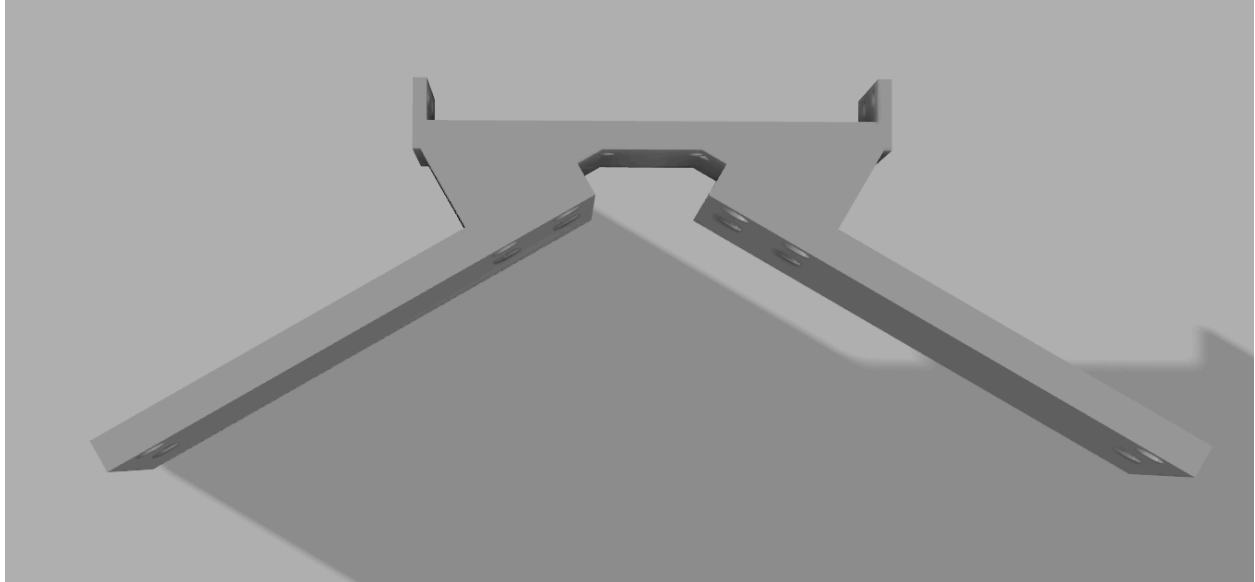


Figure 3: Rear View of Cube Capturing Mechanism

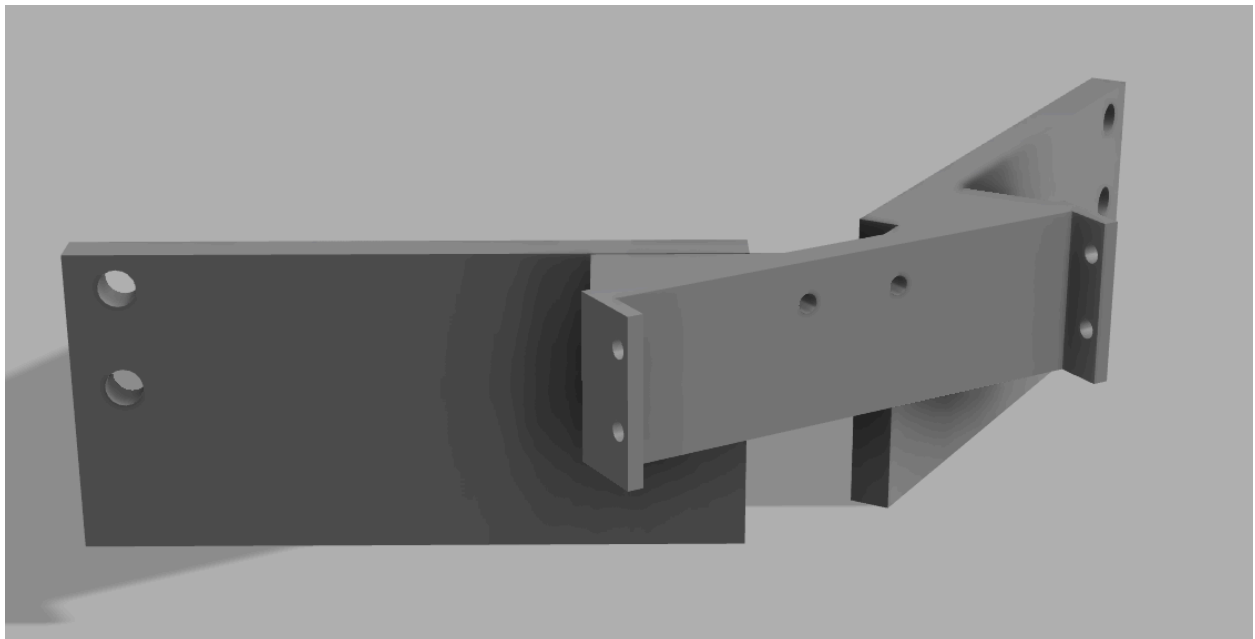


Figure 4: Acrylic Arms (Laser Cut)

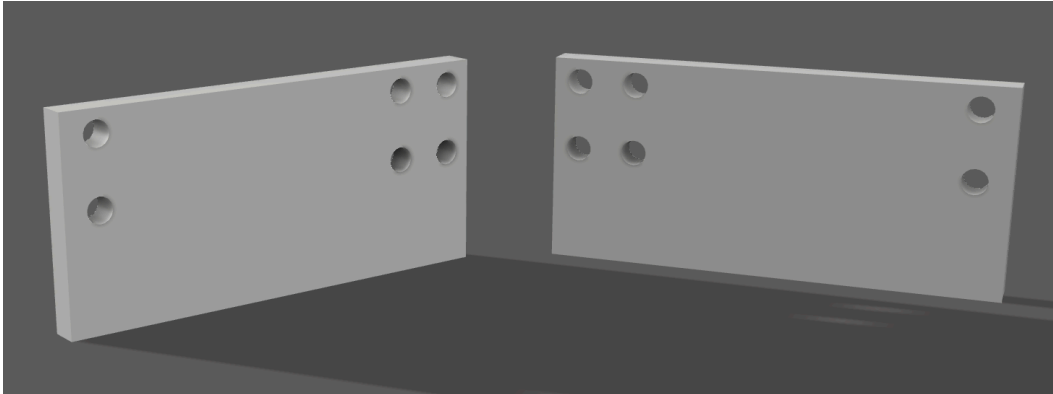


Figure 5: Mounting Bracket (3D Printed)

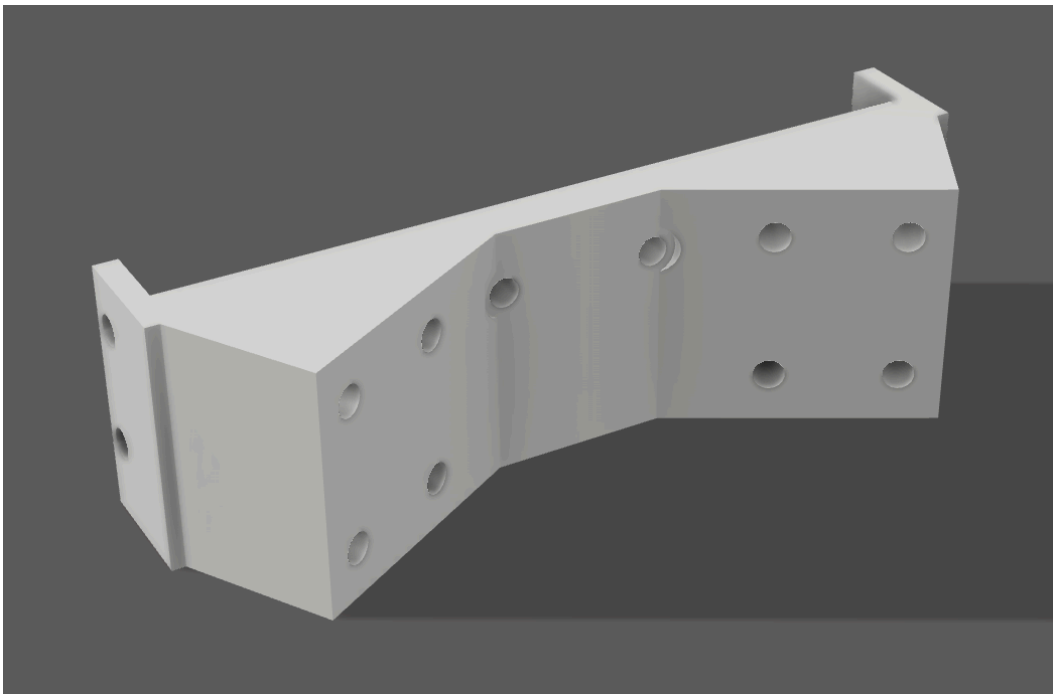
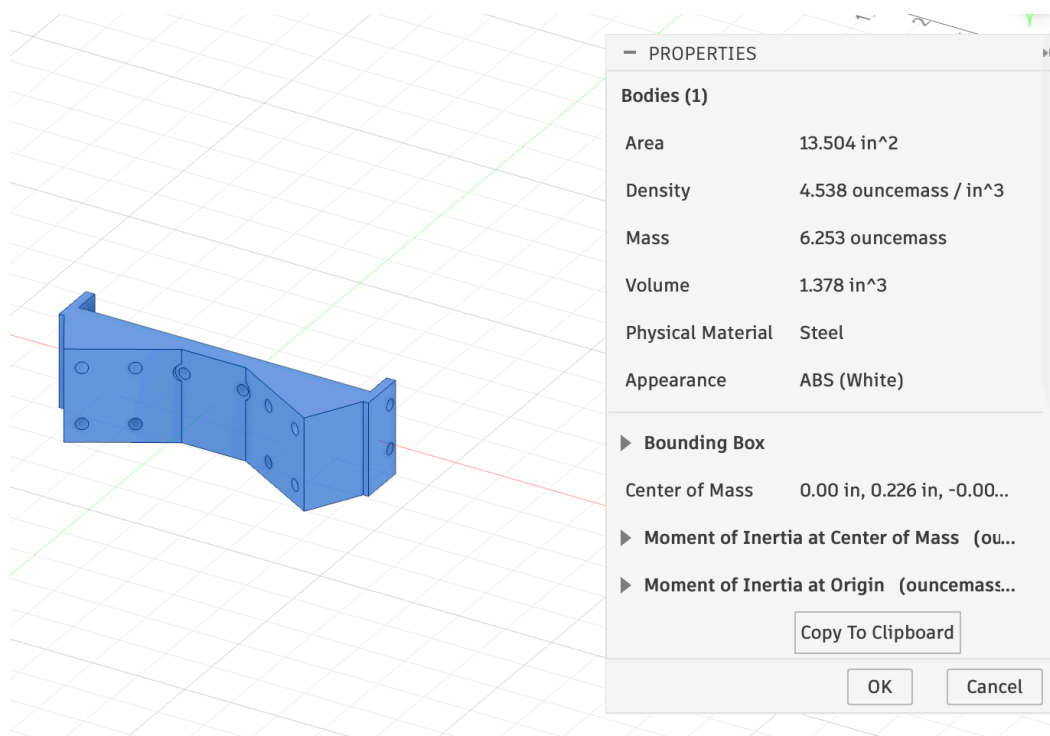
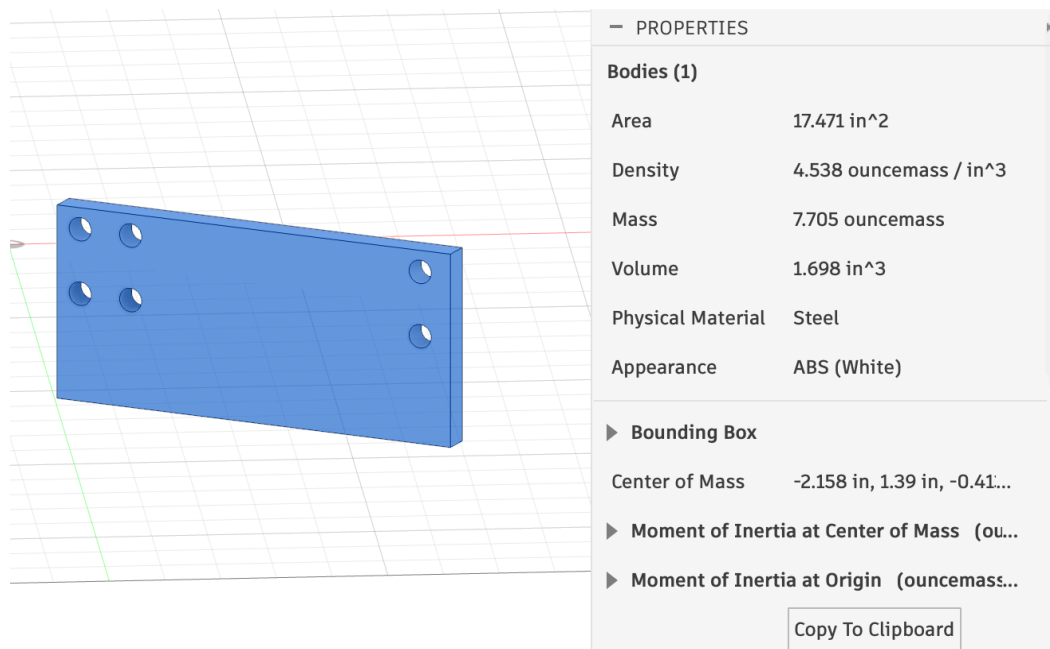


Figure 7: Mounting Bracket Properties**Figure 8: Acrylic Arm Properties**

Weight Calculations:

$$\rho_{ABS} = 1.05 \text{ g/cm}^3$$

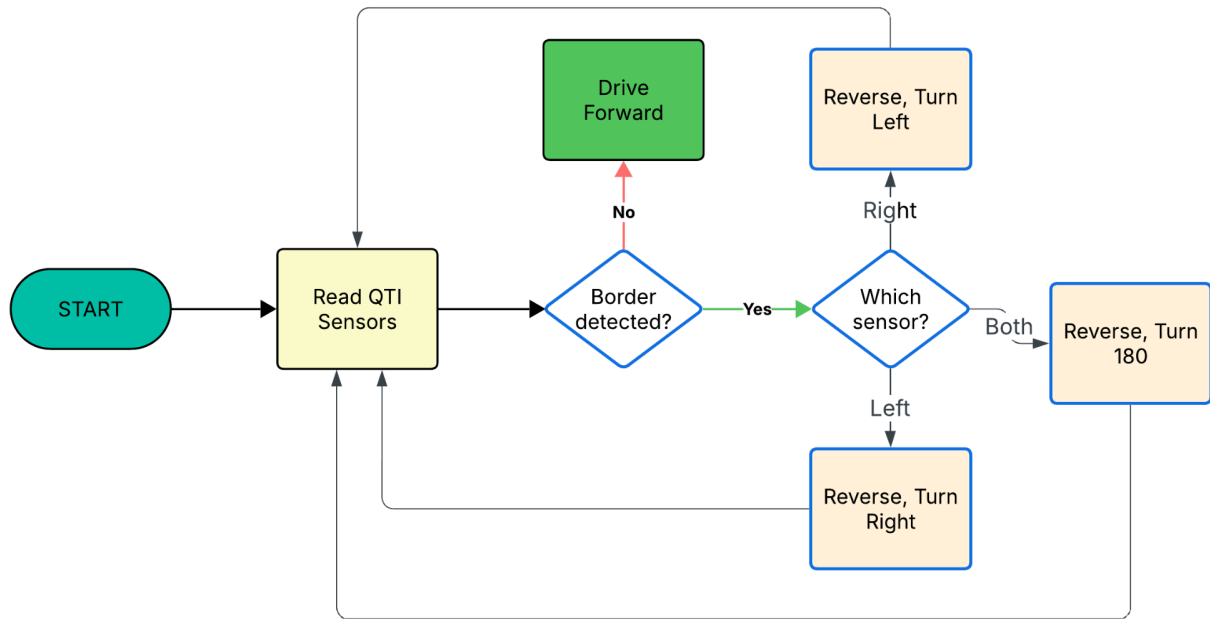
For the mounting bracket:

$$m_{\text{bracket}} = \rho_{ABS} V_{\text{bracket}} = (1.05 \text{ g/cm}^3)(1.378 \text{ in}^3)(16.387 \text{ cm}^3/\text{in}^3) = 23.71 \text{ g}$$

$$\text{Cost}_{\text{bracket}} = \$1.00 + (\$0.50/\text{g})(23.71 \text{ g}) = \$12.86$$

For the arms:

$$\text{Total Cost} = (\$0.05/\text{inch})(11.5 \text{ inches}) + (\$0.50/\text{part})(2 \text{ parts}) + \$5/\text{sheet} = \$6.58$$

Appendix D: Flowchart

Appendix E: Code

```
// ----- INITIAL FUNCTIONS -----

// Configures Arduino digital pins 8-11 (PORTB bits 0-3) as outputs for motor control.
// Setting a bit to 1 makes it an output, 0 makes it an input.
void initMotors() {
    DDRB = DDRB | 0b00001111; // Set bits 0-3 of DDRB to 1 (output),
    // leaving bits 4-7 unchanged
    PORTB = PORTB & 0b11110000; // Set bits 0-3 of PORTB to 0 (LOW),
    // ensuring motors are off at startup
}

// Configures Arduino digital pins 6 and 7 (PORTD bits 6-7) as inputs for QTI sensors.
// Bits 6 and 7 of DDRD are set to 0 (input).
// Bits 6 and 7 of PORTD are set to 0 (no internal pull-up resistor,
// since external 10kΩ pull-up resistors are used on the breadboard).
void initQTI() {
    DDRD = DDRD & 0b00111111; // Set bits 6-7 of DDRD to 0 (input),
    // leaving bits 0-5 unchanged
    PORTD = PORTD & 0b00111111; // Set bits 6-7 of PORTD to 0 (disable
    // internal pull-ups on pins 6 and 7)
}

// ----- MOTOR FUNCTIONS -----

// Each motor is controlled by two PORTB pins via an L9110H H-bridge.
// The H-bridge translates two logic signals into a current direction through the motor:
// Pin A HIGH, Pin B LOW : motor spins forward
// Pin A LOW, Pin B HIGH : motor spins backward
// Both LOW : motor stops
//
// Pin assignments:
// Bit 0 (pin 8) - Right motor, backward
// Bit 1 (pin 9) - Right motor, forward
// Bit 2 (pin 10) - Left motor, forward
// Bit 3 (pin 11) - Left motor, backward

// Drives both motors forward.
// Sets bits 1 and 2 HIGH (forward pins for right and left motors),
// and ensures bits 0 and 3 are LOW (backward pins off).
void drive_forward() {
    PORTB |= 0b00000110; // Set bits 1 and 2 HIGH
    PORTB &= 0b11110110; // Set bits 0 and 3 LOW, leave others unchanged
}
```

```

// Drives both motors backward.
// Sets bits 0 and 3 HIGH (backward pins for right and left motors),
// and ensures bits 1 and 2 are LOW (forward pins off).
void drive_back() {
    PORTB |= 0b00001001; // Set bits 0 and 3 HIGH
    PORTB &= 0b11111001; // Set bits 1 and 2 LOW, leave others unchanged
}

// Turns the robot left by driving the left motor backward
// and the right motor forward.
// Sets bits 0 and 2 HIGH, ensures bits 1 and 3 are LOW.
void drive_left() {
    PORTB |= 0b00000101; // Set bits 0 and 2 HIGH
    PORTB &= 0b11110101; // Set bits 1 and 3 LOW, leave others unchanged
}

// Turns the robot right by driving the right motor backward
// and the left motor forward.
// Sets bits 1 and 3 HIGH, ensures bits 0 and 2 are LOW.
void drive_right() {
    PORTB |= 0b00001010; // Set bits 1 and 3 HIGH
    PORTB &= 0b11111010; // Set bits 0 and 2 LOW, leave others unchanged
}

// Stops both motors by setting all motor control pins (bits 0-3) LOW.
void drive_stop() {
    PORTB &= 0b11110000; // Set bits 0-3 LOW, leaving bits 4-7 unchanged
}

// ----- BORDER ESCAPE -----

// Reads both QTI sensors and executes an escape maneuver based on
// which sensor detected the black border.
void escape_border() {

    // Read bit 7 of PIND (pin 7, left QTI sensor).
    // A non-zero result means the sensor is reading HIGH (black surface detected).
    int left_black = (PIND & 0b10000000) != 0;

    // Read bit 6 of PIND (pin 6, right QTI sensor).
    int right_black = (PIND & 0b01000000) != 0;

    if (left_black && right_black) {
        // Both sensors detect the border -- robot has hit the border head-on.
        // Reverse for 400ms, then turn left for 600ms to execute SHARP turn.
        drive_back();
    }
}

```

```

    _delay_ms(400);
    drive_left();
    _delay_ms(600);
}
else if (left_black) {
    // Only the left sensor detects the border.
    // Reverse briefly, then turn right to angle away from the border.
    drive_back();
    _delay_ms(200);
    drive_right();
    _delay_ms(250);
}
else if (right_black) {
    // Only the right sensor detects the border.
    // Reverse briefly, then turn left to angle away from the border.
    drive_back();
    _delay_ms(200);
    drive_left();
    _delay_ms(250);
}
}

// ----- MAIN -----

int main(void) {

    initMotors(); // Configure motor pins as outputs and set them LOW
    initQTI();    // Configure QTI sensor pins as inputs

    _delay_ms(2000); // 2-second startup delay to allow the robot to be
                    // placed on the field before operation begins

    // Infinite main loop: runs for the duration of the competition
    while (1) {

        // Read both QTI sensors on each iteration of the loop
        int left_black = (PIND & 0b10000000) != 0; // HIGH = black border on left
        int right_black = (PIND & 0b01000000) != 0; // HIGH = black border on right

        if (left_black || right_black) {
            // If either sensor detects the border, execute the escape routine
            escape_border();
        } else {
            // Otherwise, drive forward and collect cubes
            drive_forward();
        }
    }
}

```

```
}  
  
return 0; // Never reached, but included for standard C compliance  
}
```